FOREST GUARD AI · TECHNICAL REPORT

# Protecting Forests. Preventing Conflict. Saving Lives.

AI-assisted wildlife monitoring, explainable GIS risk, guard response and a privacy-safe community portal.

Hackathon build · 11 November 2026

Single-node · CPU inference · synthetic demo dataset

Real local inference. Seeded observations are synthetic and labelled.

REAL INFERENCE · NO LIVE FEEDS · NO GPS CLAIMS



01 · OVERVIEW

# Executive summary

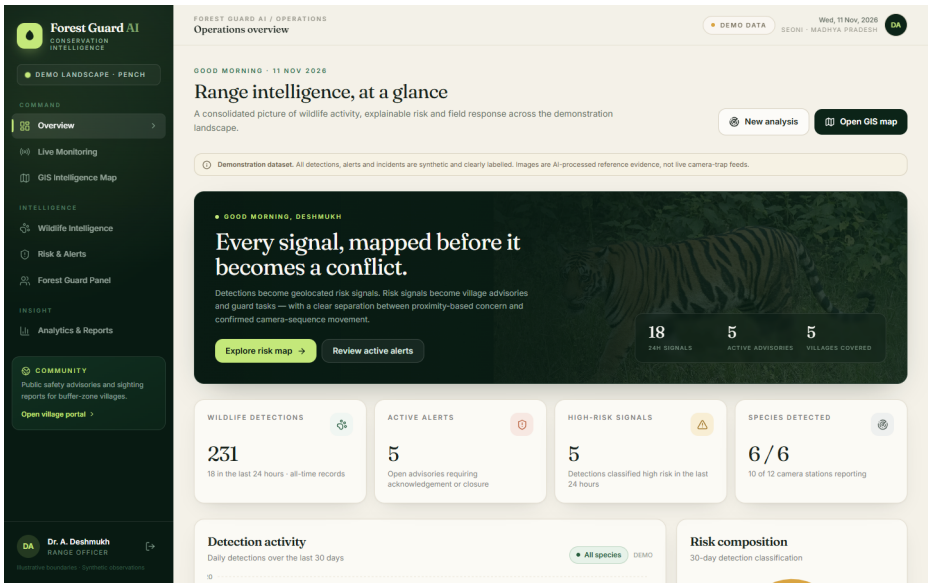
Forest Guard AI is a working, local-first platform for human–wildlife conflict prevention. A detection is analysed on the machine, attached to a real camera station, scored against village boundaries with an explainable model, converted into an assigned guard task, and republished to the public as a coarse advisory that never reveals the animal's location. It was built for a solo-student hackathon, and every screen is wired to a live API.

The honest boundary matters. Villages, cameras, guards, detections, incidents and alerts in the shipped dataset are **synthetic and labelled**. Uploaded images and short videos are **genuinely analysed** by YOLO-World v2 and CLIP on CPU. No live feeds, GPS collars, government systems or cloud inference are involved, and no field accuracy is claimed.

| Area | What was delivered | Evidence |
|---|---|---|
| Platform | FastAPI + SQLAlchemy/SQLite API; React 18 + Vite SPA with eight pages | 28 API/domain tests; production build |
| AI | Two-stage local detection (YOLO-World v2 + CLIP ViT-B/32), image and 5-frame video | Genuine upload captured in screenshot 02 |
| GIS | Leaflet map with 8 layers, clustering, filters, detail panel and risk cells | Screenshot 03; map payload tests |
| Risk | Configurable weighted score with five evidence components and species multipliers | Risk diagram; boundary tests |
| Response | Alert lifecycle, guard assignment, RBAC and append-only audit events | Lifecycle test; screenshot 05 |
| Community | Public advisories, reports with optional image, reference-code status check | Sanitisation tests; screenshot 07–08 |
| Demo data | 5 villages, 12 cameras, 6 guards, 230 detections, 18 tracks, 13 alerts, 6 incidents | Deterministic seed on first boot |

**STATUS AT A GLANCE**

Built and verified: all six modules, eight pages, 28 backend tests, production bundle, 10 live screenshots and 3 architecture diagrams. Not built or not claimed: live camera integration, GPS tracking, real SMS/email dispatch, field accuracy, scale-out infrastructure.




*The operations console at a glance. Figures on every page come from the same seeded dataset, and the demo-data badge is visible throughout.*

02 · CONTEXT

# The problem and the design response

Forest teams already collect camera images, patrol notes, incident logs and village calls. The binding constraint is not a lack of data — it is the time and judgement needed to turn fragmented records into one defensible action.

Four operational gaps shaped the product: sightings arrive late; maps and records live in different tools; warnings are broadcast to everyone or no one; and community reports disappear without a trace. Each maps to a module rather than a dashboard widget.

## Design principles

| Principle | How it is enforced |
|---|---|
| Honesty | Synthetic records carry demo badges and API flags. Uploads return a real result or a 503 — never a fabricated one. |
| Explainability | Every alert stores its score components, species weight, distance basis and a written reason. |
| Privacy by architecture | Public payloads are sanitised server-side; coordinates and camera codes never reach the browser. |
| Local-first | No cloud inference or external data calls. OpenStreetMap is used only for basemap tiles. |
| Reproducibility | A fixed random seed generates the same landscape and history on every reset. |
| Honest movement | Tracks require ≥3 detections across ≥2 cameras within 24 h and 7 km; GPS is never implied. |

## Explicit non-goals

- No live camera feeds, CCTV integration or government surveillance access.
- No GPS telemetry, animal collars or individual re-identification.
- No real dispatch channel: guard notification is simulated in-app and labelled.
- No claim of field accuracy, deployment, adoption or measured impact.
- No public exposure of sensitive wildlife locations.

**THE CORE IDEA**

A nearby sighting and a confirmed approach are different evidence. The product shows proximity, recency, frequency, incidents and movement separately, so a high score means “verify this soon” — not “a conflict will occur”.

## The evidence chain

| Stage | Question answered | Stored artefact |
|---|---|---|
| Frame | What was seen, and how sure is the model? | Annotated media, species, confidence, bounding box |
| Place | Which station and how far from a village? | Camera coordinates, boundary distance |
| Risk | Why is it urgent? | Components, weights, score, level, written basis |
| Alert | Who should act, and where? | Affected villages, assigned guard, timestamp |
| Response | What happened? | Acknowledge/respond/resolve events with actor and note |

03 · ENGINEERING

# System architecture

A React single-page client talks to a FastAPI JSON API. The API owns authentication, domain services, media serving and CSV export. The AI engine is loaded lazily on the first upload and stays resident. SQLite is the database of record; the study area, village boundaries and risk configuration are files in the repository.

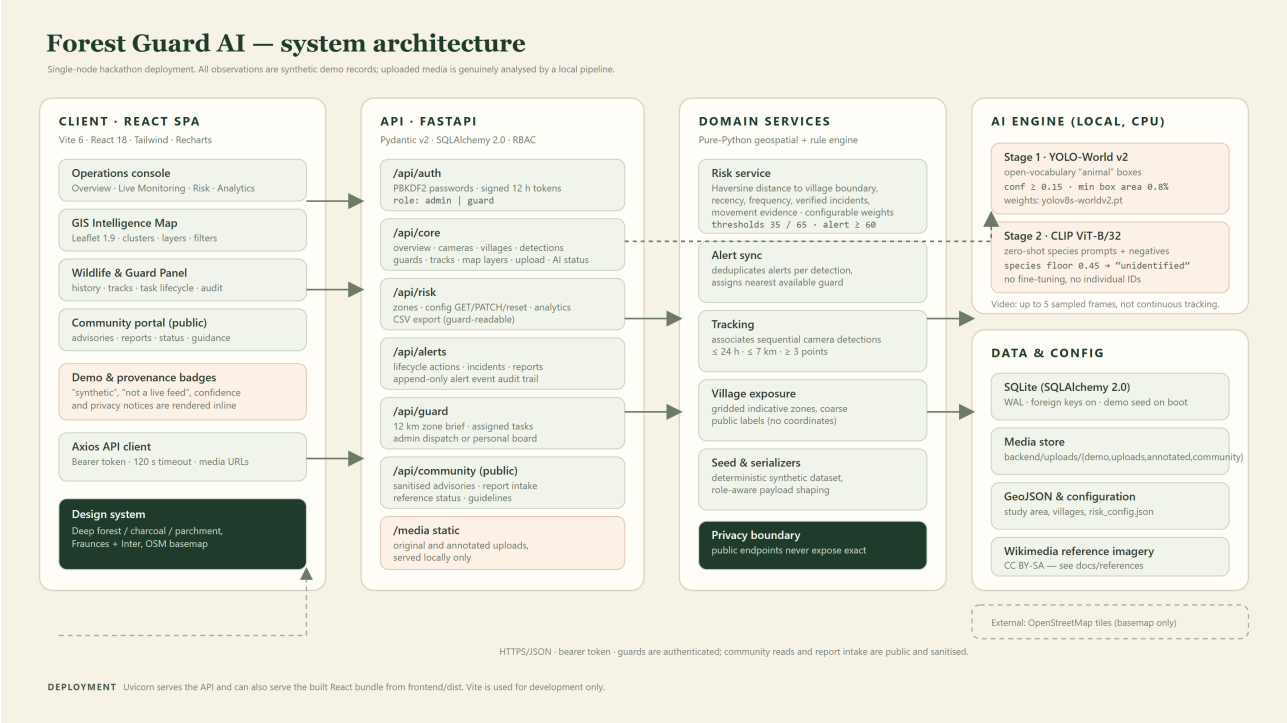



*Figure 1 — Single-node architecture. The public portal and the console read the same data through different serializers.*

| Layer | Technology | Responsibility |
|---|---|---|
| Client | React 18 · Vite 6 · Tailwind · Leaflet 1.9 · Recharts | Eight pages, map layers, forms, filters, charts |
| API | FastAPI · Pydantic v2 · SQLAlchemy 2.0 | Auth, RBAC, validation, domain endpoints, CSV export |
| Domain | Pure-Python services | Risk scoring, alert sync, tracking, seeding, serializers |
| AI | PyTorch · YOLO-World v2 · CLIP ViT-B/32 | Localisation, species scoring, annotation |
| Data | SQLite (WAL) · GeoJSON · media store | Records, boundaries, risk config, uploads |

Routers stay thin and services own the logic. The privacy boundary is implemented in serializers, not in the client: public endpoints physically omit coordinates, camera identifiers and detection media. Tests assert the absence of those keys, so a future UI change cannot accidentally leak them.

04 · MODULE 1

# AI detection: local, conservative, bounded

The engine uses two off-the-shelf models on CPU, with no fine-tuning. YOLO-World v2 is configured with the single open-vocabulary class *animal*; each proposed crop is then scored by CLIP against three prompts per species for six regional species, plus negative prompts for empty forest, vehicles and people. If the negative-prompt mass dominates, the crop is vetoed to *unidentified* — a wrong dangerous-species label is worse than an honest "verify this frame".

| Decision rule | Default | Effect |
|---|---|---|
| Detector confidence | 0.15 | Boxes below the floor are dropped |
| Minimum box area | 0.8 % of frame | Rejects tiny spurious boxes |
| Species floor | 0.45 | Weak crops become "unidentified", not a forced species |
| Combined floor | 0.50 | 0.55 × species + 0.45 × detector |
| Full-frame path | 0.62 | Separate floor when a crop fills the frame |
| Box cap | 6 per frame | After IoU de-duplication at 0.55 |
| Video sampling | up to 5 frames | Independent frames — no temporal tracking |

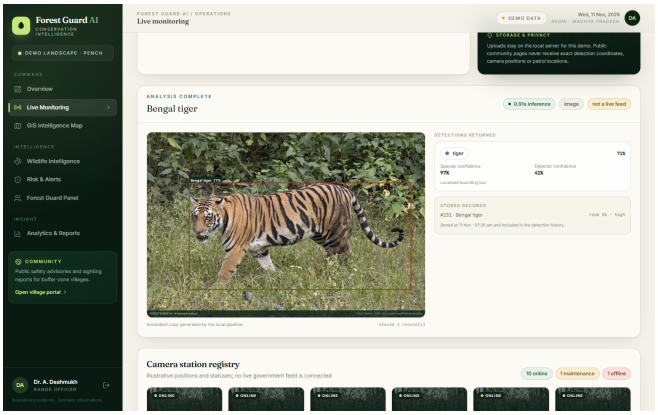



*Figure 2 — A genuine tiger upload in the running app: annotated result, 71 % species confidence, stored recognition #231, risk 86 (high).*

## What the evaluation does and does not show

| Reference image | Expected | Predicted | Detector | Species | Overall |
|---|---|---|---|---|---|
| tiger.jpg | tiger | tiger | 0.41 | 0.97 | 0.71 |
| leopard.jpg | leopard | leopard | 0.45 | 1.00 | 0.75 |
| elephant.jpg | elephant | elephant | 0.66 | 1.00 | 0.85 |
| bear.jpg | sloth bear | sloth bear | 0.80 | 0.99 | 0.90 |
| boar.jpg | wild boar | wild boar | 0.59 | 1.00 | 0.81 |
| deer.jpg | deer | deer | 0.62 | 1.00 | 0.83 |

**6 / 6 TOP-1 IS A SMOKE CHECK, NOT AN ACCURACY CLAIM**

One licenced photograph per class, no train/test split, no night or IR strata. It proves the plumbing works end to end. A defensible evaluation requires a licenced camera-trap dataset with location-based splits; see the limitations section.

05 · MODULES 2 & 3

# GIS intelligence and the explainable risk score

The map is the operational core. Forest and village polygons, camera stations, guard posts, clustered detections, gridded risk cells, alert coverage and inferred tracks are independent layers with filters, a legend and a click-through detail panel. Distance is measured to the nearest village **boundary**, not its centroid, so an edge-of-forest sighting is treated as genuinely close.

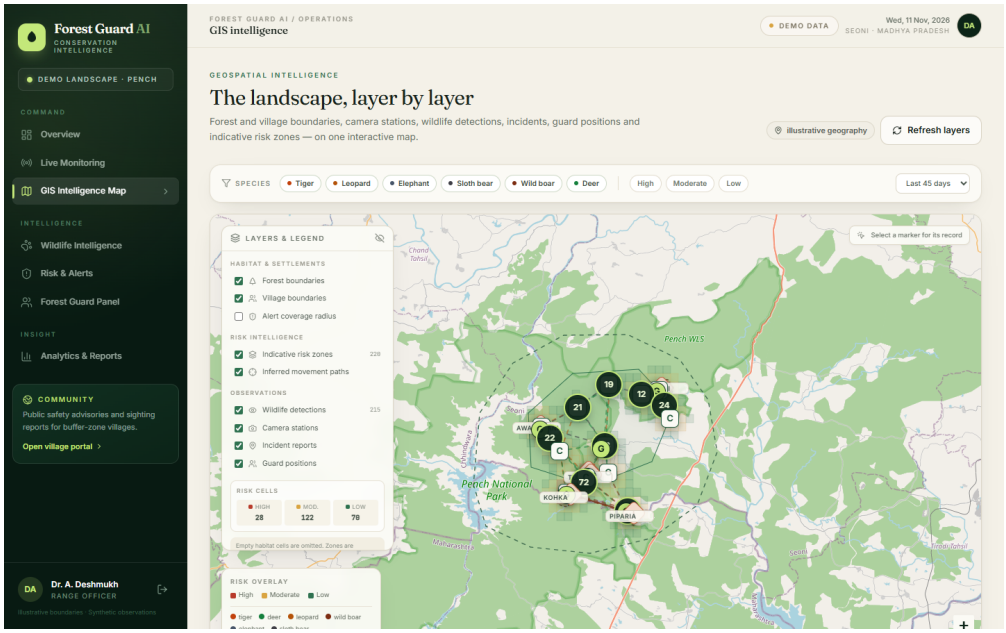



*Figure 3 — The live GIS page: layer controls, species/risk/time filters, 222 detections in the 21-day window, and risk cells with inline scores.*

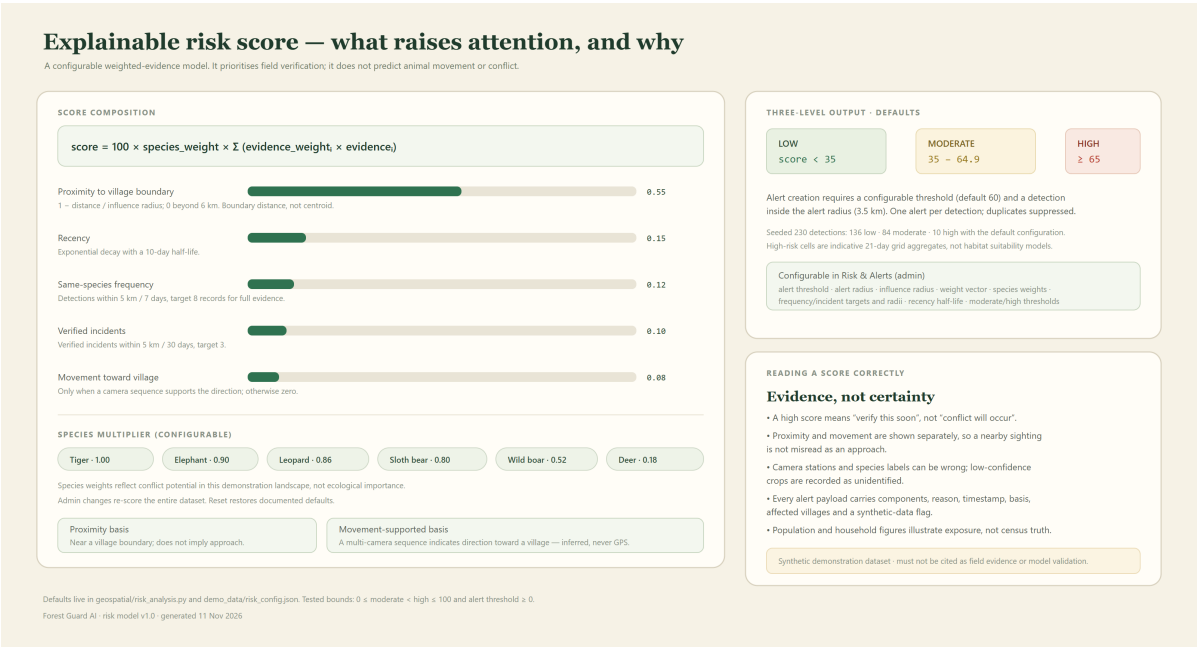



*Figure 4 — Risk model: five weighted evidence components, species multipliers, three output levels and configurable alert rules.*

Movement is a separate component worth 0.08, non-zero only when a camera sequence (≤24 h, ≤7 km, ≥3 points, ≥2 cameras) supports a direction toward a village. Tracks are labelled as inferred camera sequences, never GPS.

06 · MODULES 4 & 5

# From alert to field response

An alert is a task, not a notification. A detection at or above the configurable threshold (default 60) inside the alert radius (3.5 km) creates exactly one alert, lists every village inside that radius, records the score basis and assigns the nearest available guard. Duplicate alerts per detection are suppressed.

| Step | Actor | What is recorded |
|---|---|---|
| Detection | System | Species, confidence, camera, timestamp, risk score and components |
| Alert | System | Threshold, radius, affected villages, reason, synthetic-data flag |
| Assign | Admin / guard | Guard id, actor, timestamp |
| Acknowledge | Guard | Ownership, optional note, append-only audit event |
| Respond | Guard | Response status, note, timestamp |
| Resolve | Guard | Closure note and final state |

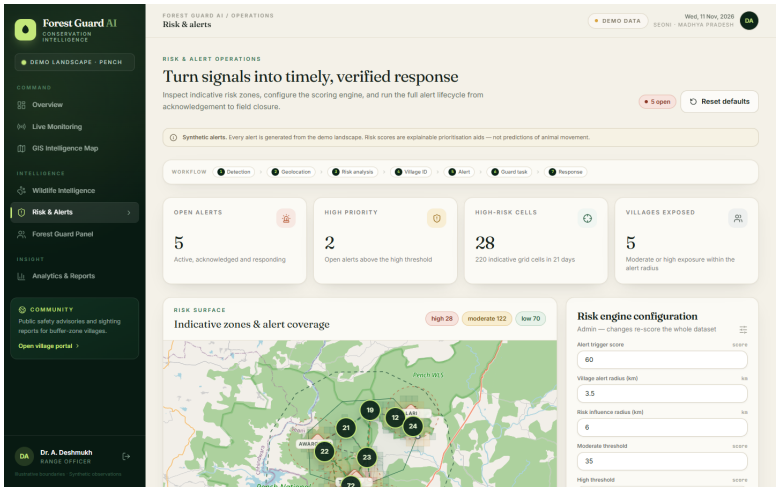



*Figure 5 — Alert queue, risk surfacing, engine configuration and the full lifecycle controls.*

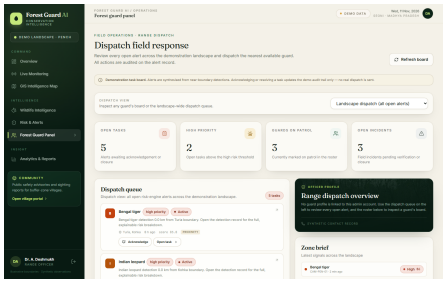



*Figure 6 — Guard board: scoped tasks, statuses and high-risk areas.*

RBAC is enforced in the API: a guard sees only their own board and cannot edit risk configuration; admins dispatch and inspect any guard. Notifications are simulated in-app — the interface says so, and no SMS or email is sent.

## Who can do what

| Role | Scope | Permitted actions |
|---|---|---|
| Admin | Entire range: all alerts, every guard board, risk configuration | Dispatch, assign, acknowledge, respond, resolve, change or reset risk config, export analytics |
| Guard | Own assigned tasks, incidents, community inbox | Acknowledge, respond and resolve own alerts; review reports and incidents |
| Public | Sanitised advisories and coarse village risk | Read guidance, submit a report, check status by reference code |

07 · MODULE 6

# Community portal and the privacy boundary

The public portal is a different view of the same data, not a separate dataset. It publishes coarse advisories by village, safety guidance, a sighting/conflict report form with an optional image, and a reference-code status lookup. All of it works without a login.

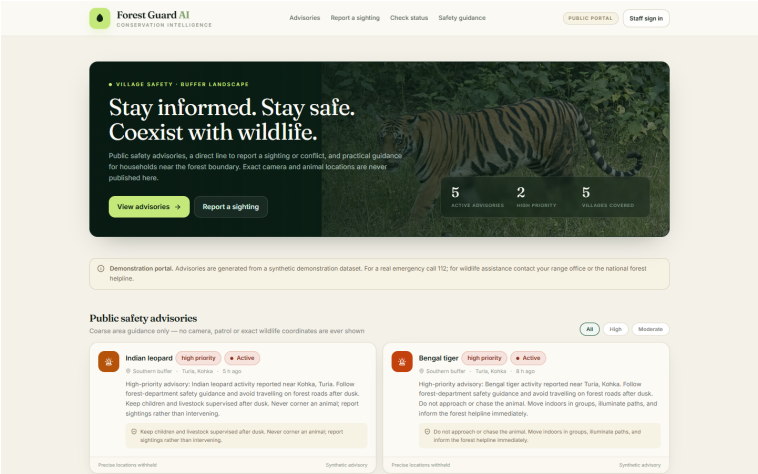



*Figure 7 — Public portal: advisories, guidance and the reporting entry point.*

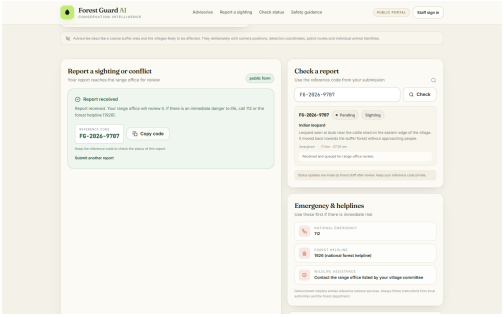



*Figure 8 — Report submission result and status check by reference code.*

## Server-side sanitisation

| Public payload | Console payload |
|---|---|
| Village names and coarse risk level | Exact coordinates and camera station codes |
| Advisory text and safety guidance | Species confidence, bounding boxes and media |
| Report reference code and status | Reporter details and moderation fields |
| Aggregated counts and trends | Guard assignment, notes and audit events |

The community API never returns latitude, longitude or camera identifiers; automated tests assert their absence. Submission validation covers category, supported species, description length, coordinate pairing and image type/size. Reference codes use the form `FG-YYYY-NNNN` and are checked for collisions.

## Public report lifecycle

- **Submit** — category (sighting or conflict), species, village, description, optional contact and photograph.
- **Validate** — supported species, description of at least 12 characters, paired coordinates, accepted image types and size.
- **Issue reference** — a collision-checked code such as `FG-2026-9707` is returned immediately.
- **Triage** — the report appears in the guard-only inbox and can be marked pending, verified or closed with a note.
- **Status lookup** — the reporter checks progress with the code, without an account and without seeing sensitive locations.

08 · QUALITY

# Verification and testing

The backend suite runs against a temporary SQLite database created per session, with the AI engine in demo mode. Tests never touch the demo database, download weights or need a GPU. The upload test asserts demo mode returns HTTP 503 and leaves record counts unchanged.

| Suite | Tests | What is verified |
|---|---|---|
| test_api_core.py | 9 | Health/meta, login and 401s, seed counts, detection filters, map layers, non-GPS tracks, demo upload 503, CSV export |
| test_api_risk_alerts.py | 5 | Analytics totals, risk zones/exposure, config RBAC + temp-file writes, full alert lifecycle audit, filters/404 |
| test_api_guard_community.py | 9 | Admin dispatch vs scoped guard board, sanitised public payloads, report round trip and validation, guard-only incidents |
| test_geospatial.py | 5 | Haversine, point-in-polygon, ring closure, threshold boundaries, explainable score bounds and weight sum |
| Total | 28 | All passed on 11 November 2026 in 1.27 s (2 unrelated deprecation warnings) |

## Other checks actually performed

- **Production build:** `npm run build` succeeds with Vite 6.
- **Browser flows:** Playwright + Edge captured all ten screenshots from the live app, including a real tiger upload and a public report submission. No browser errors were reported.
- **Data consistency:** the upload precedes the dashboard captures, so the same 231 detections and 14 alerts appear on the overview, guard panel and analytics pages.
- **AI smoke evaluation:** the production engine was run over the six reference photos; raw JSON is stored in `docs/references/`.
- **Configuration safety:** risk-config tests write to a temporary path; the repository's `risk_config.json` was verified unchanged.

Not covered: UI component unit tests, load/soak testing, accessibility audit, security penetration testing and evaluation on a labelled camera-trap dataset. These are explicit gaps, not implied passes.

## Deliverable verification

| Artefact | Location | Status |
|---|---|---|
| Backend suite | tests/ | 28 passed in 1.27 s; isolated temp database |
| Production bundle | frontend/dist/ | Vite 6 build completed |
| Screenshots | docs/screenshots/ | 10 PNG files captured from the running app |
| Diagrams | docs/diagrams/ | 3 hand-authored SVGs rendered to PNG at 2× |
| AI smoke output | docs/references/ai_smoke_evaluation.json | Recorded 11 Nov 2026 with caveats |
| Report sources | deliverables/build_deck.py · build_report.py | Reproducible from repository assets |

09 · CLOSE

# Limitations, roadmap and how to run it

| Area | Current limit | Consequence |
|---|---|---|
| Classifier | Zero-shot CLIP, six labels, no fine-tuning | Will confuse similar species and struggle with night/IR frames |
| Evaluation | Six reference photos, one per class | No accuracy or field-performance claim is possible |
| Storage | SQLite, no spatial index | Single-node only; polygons parsed in Python |
| Tracking | Camera-sequence association, no individual IDs | Movement is a coarse, inferred direction |
| Dispatch | Simulated in-app notification | No real responder SLA or SMS delivery |
| Landscape | Illustrative Seoni-area geometry and synthetic population | Exposure figures illustrate the method, not census truth |

## Next steps, in priority order

- **1 · Measurement.** Licence a regional camera-trap dataset, split by camera location, and report per-species precision/recall at a fixed threshold, stratified by day/night and occlusion.
- **2 · Spatial infrastructure.** Move to PostgreSQL + PostGIS with spatial indexes, then add a task queue so inference scales beyond one process.
- **3 · Real response.** Integrate an SMS/radio gateway with delivery receipts and acknowledgement SLAs; add an offline-first guard client.
- **4 · Governance.** Add audit export, data-retention rules and role-scoped media access before any pilot deployment.

## Reproduce the build

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m uvicorn backend.main:app --port 8000

cd frontend && npm install && npm run dev
python -m pytest -q
```

Open `http://127.0.0.1:5173` and sign in as **admin@forestguard.ai / forest123** (admin) or **guard@forestguard.ai / guard123** (guard). The community portal is public at `/community`. Setup, API, testing, demo script, AI limits and image credits live in `docs/`.

Imagery is CC BY-SA / CC BY-SA-compatible from Wikimedia Commons and individually attributed in `docs/image-credits.md`; basemap © OpenStreetMap contributors. Model weights are excluded from the repository and must be obtained separately. This report describes a demonstration system, not a validated or deployed service.